medAIval

Stanford CS229 Project

Daniel Yang
Department of Computer Science
Stanford University
dy92634@stanford.edu

Thomas Shundi
Department of Mathematics
Stanford University
tshundi@stanford.edu

Ethan Farah
Department of Mechanical Engineering
Stanford University
efarah@stanford.edu

1 Abstract

Accurate ICD-10 coding from clinical notes is a critical process for hospitals to maintain, both from a billing and insurance perspective as well as for patients in terms of maintaining consistency across different hospital systems. The process is not only tedious and costly for a hospital, but also prone to human error, especially with ever-evolving iterations of the ICD system [1]. We study fully automated ICD-10 coding for sinus-related diagnoses using clinical NLP techniques. We developed and evaluated several approaches on 4,400 de-identified clinical notes from the Stanford Ears, Nose, and Throat (ENT) Clinic, including (1) binary relevance classifiers per code, (2) a deep model combining ClinicalBERT with an LSTM and attention mechanism, (3) embedding models with similarity classifiers, and (4) a two-tower embedding that fuses the branches with label-specific attention layers and synonym enhancement neural network layers. Experiments show our models on par and occassionally beating previous benchmarks. Our binary classifiers (F1 Score $\approx$0.79) and the two-tower model (Accuracy $\approx$0.94) had notable quantitative performances. We report precision, recall, F1, and analyze loss functions (binary cross-entropy, focal loss, and asymmetric loss) and overfitting mitigation. Our results demonstrate that domain-specific pre-trained language models can significantly improve automated coding of sinus diagnoses. We discuss challenges such as class imbalance and note variability, and outline future improvements with larger datasets, hierarchical coding, and integration of advanced language models.

2 Introduction

The *International Classification of Diseases, 10th Edition (ICD-10)* provides a standardized taxonomy for diseases and procedures [1]. This system is important because it enables consistent documentation for billing and research [1]. In practice, manual "coders" are responsible for logging the ICD-10 code associated with a patient's visit's outcome; however, especially among codes which are similar or highly related, this manual-logging process becomes more error prone, time-consuming, and costly. We add on a special challenge to automating ICD coding by analyzing strictly sinus clinic notes. This is more challenging because these codes are very similar, and in some cases nearly identical (i.e. J01.10: Acute frontal sinusitis, unspecified vs J01.11 Acute recurrent frontal sinusitis). Performing this task accurately is valuable, as doing so could drastically reduce administrative burden and improve accuracy in billing. However, *clinical NLP challenges* such as unstructured text, abbreviations, and varying levels of detail impede automatic coding.

Each patient's clinical notes may contain multiple diagnoses, aking this a multi-label classificatio problem. Additionally, models must handle class imbalances, as some sinus diagnoses are much more common than the extremely rare ones in the dataset. In this paper, we focus on automating ICD-10 coding for sinus-related conditions using NLP, evaluating a range of modeling approaches. We hypothesize that a domain-tuned language model with sequence modeling and attention can better capture the nuance in sinus clinic notes, leading to improved coding performance. We also emphasize **HIPAA compliance**: all patient notes were de-identified (removal of names, dates, etc.) to protect privacy. The following sections describe our dataset, methods, experiments, and results, concluding with future directions for clinical coding automation.

3 Related Work

Fully automated medical coding has been an active area of clinical NLP research for over a decade. Approaches have been in a couple of different ways: from applying machine learning with rule-sets or nlp-ideas [2], to using recurrent neural networks (RNNs) to encode clinical text for ICD code prediction [3]. These methods have been expanded on with the development of deep learning and convolutional neural networks with the development of CAML (Convolutional Attention for Multi-Label), specifically looking at assigning ICD codes from ICU discharge summaries [4]. Extensions like DR-CAML improved the model by adding an additional encoder for ICD code descriptions, basically giving the model access to a thesaurus, and this provided a signiicant improvement over the CAML model [2]. Given the hierarchical structure of ICD codes, several works have tried to caputre this relationship in their model construction. For example, one approach used a desenely connected CNN with graph CNN to capture the hierarchical structure [5]. Lastly, the most recent works have attempted to finetune LLMs such as BERT by using multi-label attention [6].

It is critical to note that almost every model we discuss in the relevant work section has been trained on the Top 50 ICD Labels from the acclaimed MIMIC III and IV datasets from MIT. We train on 21 near-identical ICD-10 codes which are largely unbalanced from an extremely raw dataset that is generally either incompletely or fully unlabeled (as it is data pulled directly from hospital records, not data that a team of researchers has formatted).

4 Dataset and Features

Through Dr. Fahed Noel Ayoub at Stanford ENT Clinic, we secured IRB approval granting access to retrospective data from approximately 4,400 patients, comprising 9 gigabytes of clinical notes, pathology reports, radiology results, lab orders, prescriptions, and diagnosis codes.

HIPAA compliance is maintained using Stanford's STRIDE NLP model, which scrubs protected health information by shifting dates, randomizing names, modifying lab results, and removing uncertain content. Each patient retains a unique ID across all note types to preserve data integrity.

Under Dr. Ayoub's guidance, we focused on classifying semantically similar diagnostic codes in the J01 (acute sinusitis) and J32 (chronic sinusitis) families. Some codes, like J01.31 (Acute recurrent sphenoidal sinusitis), are exceptionally rare with fewer than 250 cases across Stanford's entire 5-million patient database.

Since diagnosis codes aren't directly linked to clinical notes, we associated them by sorting records by patient and date, utilizing the fixed date-shift to match clinical notes with subsequent diagnoses.

Each patient averages over 150 text documents. During preprocessing, we excluded fully scrubbed notes and those under 50 words. We implemented two preprocessing approaches:

1. Text cleaning: removing excess whitespace, standardizing to lowercase, eliminating stopwords, and tokenizing with BioClinicalBERT to generate 768-dimensional embeddings.
2. UMLS implementation: utilizing the 33GB Unified Medical Language System to map medical terms to associated ICD-10 codes [8]. This revealed 7,685 codes referenced in our dataset, enabling us to construct sparse vectors representing ICD-10 codes appearing in each patient's clinical notes.

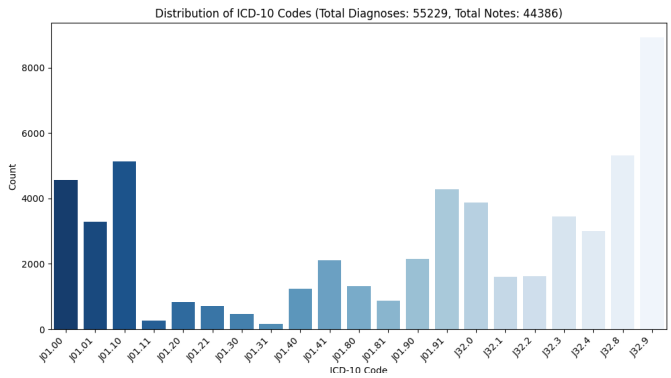

Figure 1: Distribution of the chosen ICD code space in our dataset

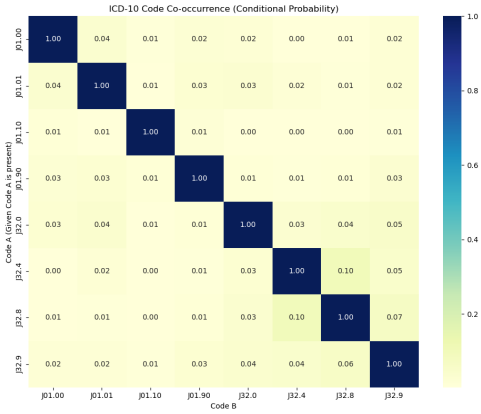

Figure 2: Co-concurrence of the top four most common J01 (acute) codes and top four most common J32 (chronic) codes

We use either a 0.8-0.2 training-validation split or a 0.7, 0.2, 0.1 training-validation-test split on all of our data when training, and in order to deal with overloading RAM locally, at times we incorporate lazy-loading datasets for our larger input files. Additionally, to deal with some of the massive discrepancies in data sizes, we randomly downsample upon generating our dataloaders.

5 Methods

Before we explore the development of our models, we must discuss how we maintained HIPAA compliance. In order to maintain airtight HIPAA compliance, every training instance was done fully locally and offline. This generally involved downloading packages like NLTK and models like BioClinicalBERT locally and running off local file paths instead of making API calls.

Now, we can get to the fun part! To tackle this challenge of mapping clinical notes to their respective ICD-10 codes, we devised three seperate methods for representing the data and trained four respective models to handle their respective data representation (and figure out who's king... hint: it's *always* LeBron).

5.1 Binary Classifiers

The first approach in assigning ICD-10 codes to input texts was treating each ICD-10 code as a seperate entity. This involved training a series of binary classifiers for each ICD-10 code. For each input text, all binary classifiers would run, each one determining if a specific ICD-10 code should be assigned or not to the input text.

$$\text{Emb}(x_i) = \text{TokenEmbed}(x_i) + \text{PositionEmb}(i) + \text{SegmentEmb}(i) \quad (1)$$

$$\mathcal{L} = -\sum_{c \in \{0,1\}} \mathbf{1}(y = c) \log(\hat{p}_c). \quad (2)$$

Each input token is first converted into a vector embedding seen in equation 1. Using pretrained weights from the Bio_ClinicalBERT word encoder, the embedding is passed through 12 transformer layers, each layer consists of multi-headed attention followed by a feed forward network with layer normalization. Finally, the hidden state is passed through a final classification head which is a fully connected layer to get logits for both classes.

Because we treat each ICD-10 code with a binary classifier, we perform cross entropy loss on the two logits where $\hat{p}_c$ is the predicted probability for the correct class 2. Our ground truth label is represented by $y \in \{0, 1\}$. We train each classifier for 2 epochs using batch sizes of 8.

5.2 Vector Encoding with LSTM

The second approach in our ICD-10 code assignment models was to represent each target ICD-10 codes as a 21-vector of probabilities. After compiling the 21 most relevant ICD-10 codes to the ENT clinic in our dataset, we assigned each code an

arbitrary but ultimately fixed index along a 21-length vector. For each clinical note, a 1 signals the respective ICD-10 code at that index being diagnosed and a 0 means no respective ICD-10 code was assigned to the clinical notes.

Our methodology involves passing input sequences through Bio_ClinicalBERT to generate word embeddings upon forward calls. These embeddings serve as input to a bidirectional LSTM (long short-term memory) network, which produces hidden states $h_1, \ldots, h_T$. Next, we apply a *label-specific multiheaded attention mechanism* over the LSTM outputs to focus on the most relevant parts of the note for coding. This is done because often only small segments of a paragraph are attributable to the presence of an ICD-10 code.

$$\alpha_t = \frac{\exp(h_t^\top u)}{\sum_n \exp(h_n^\top u)} \quad (3) \qquad\qquad v = \sum_t \alpha_t h_t \quad (4)$$

We use a trainable context vector for each output code or a general vector u to compute attention weights α_t across token positions. For each hidden state, we find e_t, a scalar representing how well h_t matches the query u. We normalize these scores and apply softmax to get our attention weights α_t 3. Finally, we compute our attended context vector v, which weights each hidden state with the respective calculated weight 4. This attention vector v should concentrate the information from the words indicative of a particular diagnosis. The attended representation v is then fed to a fully-connected Dense layer and a sigmoid output layer that produces a probability for each ICD-10 code.

Since our output is a vectorized format for the presence of ICD-10 codes, we used Binary Cross Entropy with Logits Loss. For any given predicted logits and the corresponding ground truth label, we apply sigmoid activation and cross entropy loss to sum across all the predictions within the ICD-10 vector.

5.3 Embedding Similarity

Our third approach in ICD-10 representation comes from the official ICD-10 code description themselves. Since each ICD-10 code is attributable to a specific set of symptoms, each ICD-10 code is converted to a short phrase. Our model works by measuring the embedding similarities between the input text and our class text.

$$p_emb = \frac{1}{N}\sum_{i=1}^{N} WH_i + b, \quad W \in R^{d\times d}, H_i \in R^{L\times d}, b \in R^d \quad (5)$$

We start with chunking our input paragraphs, because they often exceed our token limit (512 subwords). For each overlapping chunk, we encode it through a projection head that involves Bio_ClinicalBERT with a trainable linear layer. We average all the chunk embeddings to get our paragraph embedding (p_emb) for this specific input 5. Our H_i is the output from our base Clinical BERT model where L is our output sequence length. For every class, which is represented as a phrase, we push the class label through the base Bio_ClinicalBERT encoder but without the trainable linear layer. These serve as our guideposts for training our linear layer.

$$\text{pos_sim} = \frac{\text{p_emb} \cdot \text{pos_emb}}{\|\text{p_emb}\|\|\text{pos_emb}\|}, \text{neg_sim} = \frac{\text{p_emb} \cdot \text{neg_emb}}{\|\text{p_emb}\|\|\text{neg_emb}\|} \quad (6)$$

$$\mathcal{L}_{\text{example}} = \max\left(0, \delta + \text{neg_sim} - \text{pos_sim}\right). \quad (7)$$

During training, we run through all the inputs in our test dataset and for each input, we source the correct label's embedding and sample a random negative label embedding. If an input has multiple ICD-10 codes attached, this process would be repeated for each correct target label. We calcualte the cosine similarity of input label to both the positive and negative class 6. Using margin-based contrastive loss, we could encourage the paragraphe embedding to be closer in cosine similarity to the positive label embedding than the negative label embedding by at least some margin δ 7. During evaluation, we calculate the paragraph embedding for the input and select the class that yields the highest cosine similarity with our paragraph embedding.

5.4 Two-Tower Integration Architecture and Training Methodology

Our training methodology employs two-tower architecture that effectively marries sparse synonym vectors with dense text embeddings, creating a robust system for ICD-10 classification [7]. This approach bridges the gap between structured medical knowledge (synonyms) and unstructured clinical notes.

$$\text{syn_features} = \text{LayerNorm}\big(\text{Linear_2}\big(\text{GELU}\big(\text{LayerNorm}\big(\text{Linear_1}(\text{syn_vec})\big)\big)\big)\big) \quad (8)$$

The first tower processes clinical notes through BioClinicalBERT and bidirectional LSTM layers to capture contextual information. Concurrently, the second tower transforms sparse synonym vectors (representing known medical terms) through a projection network (equation 8) to align them with the text embedding space.

$$\text{attn_scores} = \frac{\text{Query}(\text{text_feat}) \cdot \text{Key}(\text{syn_feat})^T}{\sqrt{d}} \quad (9)$$

Rather than simply concatenating these representations, we implement a cross-attention mechanism (equation 9) where the text features attend to the synonym features. This creates a dynamic, context-aware integration that emphasizes relevant medical concepts based on their presence in the note.

$$\text{logits} = \alpha \cdot \text{label_specific_logits} + (1-\alpha) \cdot \text{global_logits} + \text{correlation_adjustment} \quad (10)$$

The training process optimizes both towers simultaneously through a dual classification approach (equation 10). Label-specific classifiers operate on contextualized representations from the text tower, while a global classifier integrates information from

both towers. A learnable parameter α balances the contribution of each classification pathway, allowing the model to dynamically adjust the importance of structured medical knowledge versus textual context.

$$\mathcal{L} = 0.5 \cdot \mathcal{L} * \text{focal} + 0.5 \cdot \mathcal{L} * \text{asymmetric} \tag{11}$$

To address the significant class imbalance in medical coding, we employ a weighted combination of Focal Loss and Asymmetrical Loss (equation 11). The Focal Loss component prioritizes difficult examples, while the Asymmetrical Loss applies different focusing parameters to positive and negative examples, effectively handling the predominance of negative cases in multi-label classification.

$$w_c = \log\left(\frac{N}{n_c}\right) \cdot \begin{cases} 2.0 & \text{if } n_c < 300 \\ 1.0 & \text{otherwise} \end{cases} \tag{12}$$

Additionally, we implement dynamic class weighting (equation 12) where rare ICD-10 codes receive substantially higher weights. This ensures that the model remains sensitive to infrequent but clinically significant conditions.

6 Findings and Results

6.1 Model Hyperparameters

We utilized an Adam Optimizer to apply an adaptive learning rate to our network for the binary classifiers and an AdamW Optimzier for the LSTM-based models. These optimizers (and their respective beginning learning rates) were chosen after a brief early-stopped full-factorial experiment involving different initial learning rates and the choice between Adam and AdamW.

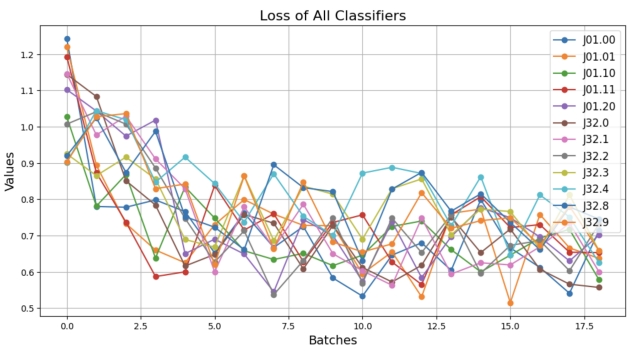

Figure 3: Binary Cross-Entropy Loss Visualization for Binary Classifiers

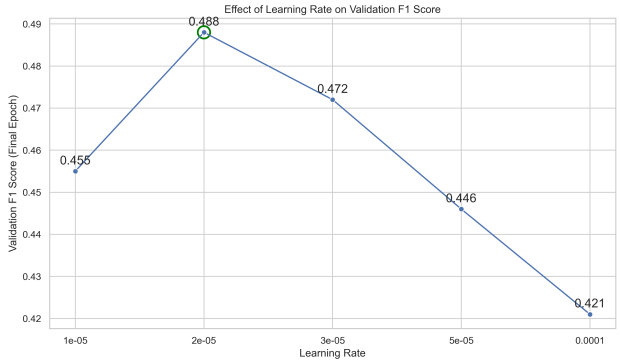

Figure 4: Experimenting on changing initial learning rate with AdamW optimizer with the two-tower model

This momentum-based update leads to faster model convergence by accelerating the model in areas with consistent gradients and maintaining previous momentum directionality to avoid getting stuck in local minima. Our initial learning rate was set small at 0.0001 for the binary classifiers and at $2(10^{-5})$ for the LSTM-based models to avoid overshooting the optimal solution. We generally use smaller batch sizes (8, 16) due to RAM limitations, with the additional benefit of better generalization and avoiding local minima solutions. The number of epochs our models were trained on ranged between 2 and 20, with the optimal per-value model being determined by simply training models with varying epoch-ranges.

6.2 Metrics and Scores

To compare and evaluate our series of models against our benchmark, we utilize F1 score, accuracy, and recall. We want to avoid false negatives (FN) over false positives (FP) when assigning ICD-10 codes to clinical notes because we want to minimize under-billing and denied insurance as much as possible. Consequently, metrics like F1 and recall, as defined in Equation (13), can capture this importance.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \qquad F_1 = 2 \times \frac{\frac{\text{TP}}{\text{TP+FP}} \times \frac{\text{TP}}{\text{TP+FN}}}{\frac{\text{TP}}{\text{TP+FP}} + \frac{\text{TP}}{\text{TP+FN}}} \tag{13}$$

We compare the test F1, accuracy, and recall scores across our four models and against previous studies, which, while related, differ in scope. Our Stanford training set is messy compared to the current state-of-the-art MIMIC dataset that these top-of-the-line models use, and our classification task is more challenging because the ICD-10 codes we classify belong to a niche field with high inter-code similarity. In contrast, prior studies benefited from a broader range of ICD codes with minimal symptom overlap.

Table 1: Micro-F1 Scores for Existing Works [9]

Model Name	Micro-F1
MSMN	.758
PLM-ICD	.755
CAML	.692

Table 2: Performance Metrics of Our Models

Model Name	F1 Score	Accuracy	Recall
Binary Classifiers	0.792 (avg)	0.779 (avg)	0.857 (avg)
Vector Encoder	0.663	0.692	0.538
Embedding Similarity	0.422	0.418	0.514
Two Towers	0.512	0.936	0.683

From the models we did create, we observe a wide spectrum of performance for F1, Accuracy, and Recall. Our results also make intuiative sense. Our binary classifiers perform better than our vector encoder and embedding similarity models. Because our binary classifiers trains a seperate model for each ICD-10 code, the aggregated model can maintain ICD-10 code specific, which translates to more flexibility and better performance at the cost of higher compute costs.

Figure 5: Attention Visualization

Our embedding similarity model performs the worst, likely because encoding a single paragraph-level embedding for the entire input loses critical information. Since ICD-10 code assignments are often triggered by just one sentence or phrase, this approach dilutes key details. Our LSTM based approach also performs better than some of our benchmarks, which is likely because of nested attention mechanisms that learn to emphasize certain segments of the inputs. Our two-tower approach also sees higher performance metrics by using two parallel models to search for strong similarities and weight importance.

6.3 Discussion

In looking at the heatmaps of two codes with higher class counts, we notice that there are more entries in the "true" diagonal of the heatmap than the "false" diagonal. It is worth discussing false negatives, false positives, and true positives, of which we have far more true positives than either other metric. Thus, we somewhat optimally solve both identifying and not overdiagnosing J codes.

Overall, our models performed on par and occassionally better in our quantitative metrics against previous studies. Qualitatively, our models performed better when they maintained custom weights for each ICD-10 code or used nested attention to focus on key segments, compared to generalized models that inferred codes from the entire input. We do not believe overfitting occurred, as we reduced clinical note variability into consistent embedding vectors. Additionally, our tuned loss functions and emphasis on recall (which penalizes false negatives heavily) helped mitigate overfitting. A model overfitting to zeros could artificially achieve high scores, so we remained cautious in our evaluation.

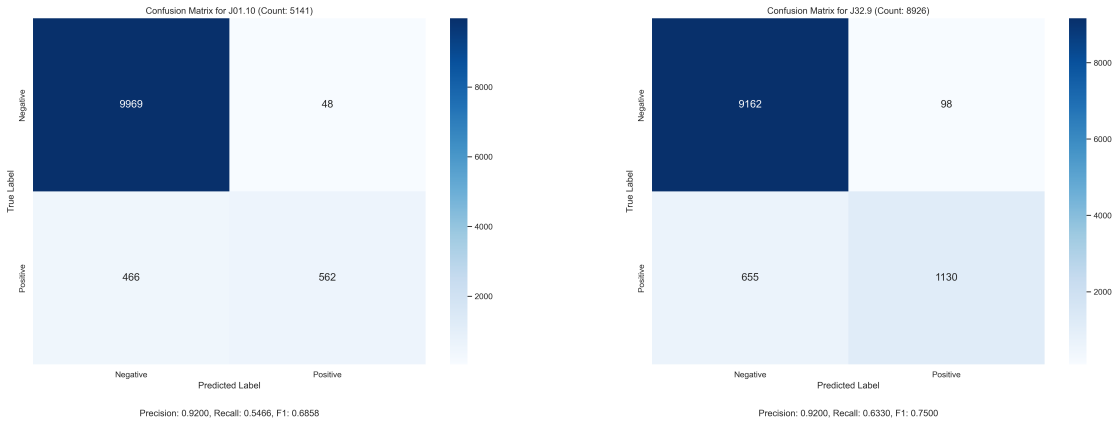

Figure 6: Confusion Matrices for Two Example ICD-10 Scores

7 Conclusion and Future Work

In this paper, we investigated the automatic assignment of ICD-10 codes for sinus-related diagnoses, with our two-tower integration architecture outperforming other approaches in terms of accuracy. This model achieved micro-averaged F1 scores of approximately 0.51, which is significant given the semantic similarity between target codes and the extreme rarity of some conditions. We also proposed a binary classifiers method which had the best F1 score of .792 (as an average over all of them) but a worse accuracy at .779. We observed that our models achieved better performance when they assigned custom weights to each ICD-10 code or used nested attention to focus on key segments, compared to generalized models that inferred codes from the entire input. Despite these achievements, our work faced challenges including extreme dataset imbalance, semantic similarity between codes, and variability in clinical documentation styles. Future research could extend this work by incorporating larger, cleaner datasets, implementing hierarchical classification to leverage parent-child code relationships, integrating advanced language models, exploring multi-modal approaches that combine notes with imaging and lab data, developing explainable AI mechanisms, implementing active learning frameworks, and incorporating temporal modeling to better distinguish between acute and chronic conditions.

8 Contributions

Ethan: Helped implement LSTM-enhanced models (lstm with multiheaded and two towers), reaching out to physician mentor, second half of data processing pipeline, wrote 1/2 methods, intro, dataset

Daniel: first half of data processing pipeline, coded out all binary classifiers and all embedding similarity models. wrote 1/2 experiments/discussions, abstract, majority of methods

Thomas: UMLS preprocessing and generating the icd synonym vector data process and models. Helped implement the two-tower method. wrote 1/2 experiments/discussions, conclusion, related works

References

1. Hirsch, J. A., et al. (2016). ICD-10: History and context. AJNR American Journal of Neuroradiology, 37(4), 596–599. https://doi.org/10.3174/ajnr.A4741
2. Moons, E., Khanna, A., Akkasi, A., & Moens, M.-F. (2020). A comparison of deep learning methods for ICD coding of clinical records. Applied Sciences, 10(15), 5262. https://doi.org/10.3390/app10155262
3. Choi, E., Bahadori, M. T., Schuetz, A., Stewart, W. F., & Sun, J. (2016). Doctor AI: Predicting Clinical Events via Recurrent Neural Networks. JMLR workshop and conference proceedings, 56, 301–318.
4. Mullenbach, J., Wiegreffe, S., Duke, J., Sun, J., & Eisenstein, J. (2018). Explainable prediction of medical codes from clinical text. In M. Walker, H. Ji, & A. Stent (Eds.), Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers) (pp. 1101–1111). Association for Computational Linguistics. https://doi.org/10.18653/v1/N18-1100
5. Xie, X., Xiong, Y., Yu, P. S., & Zhu Y. (2019). EHR Coding with Multi-scale Feature Attention and Structured Knowledge Graph Propagation. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM 19). Association for Computing Machinery, New York, NY, USA, 649–658. https://doi.org/10.1145/3357384.3357897
6. Zhang, Z., Liu, J., & Razavian, N. (2020). BERT-XML: Large scale automated ICD coding using BERT pretraining. In A. Rumshisky, K. Roberts, S. Bethard, & T. Naumann (Eds.), Proceedings of the 3rd Clinical Natural Language Processing Workshop (pp. 24–34). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.clinicalnlp-1.3
7. Google Cloud. (2023, October 26). Scaling deep retrieval with TensorFlow Two-Towers architecture. Google Cloud Blog. https://cloud.google.com/blog/products/ai-machine-learning/scaling-deep-retrieval-tensorflow-two-towers-architecture
8. U.S. National Library of Medicine. (n.d.). Unified Medical Language System (UMLS). National Institutes of Health. https://www.nlm.nih.gov/research/umls/index.html
9. Papers with Code. (n.d.). Medical code prediction on MIMIC-IV (ICD-9) - State of the Art. Retrieved from https://paperswithcode.com/sota/medical-code-prediction-on-mimic-iv-icd9